



КАФЕДРА Системы обработки информации и управления

Тема: «Алгоритмы Actor-Critic»

1

1) Введение

Обучение с подкреплением является одним из направлений машинного обучения, в котором агент обучается принимать решения на основе взаимодействия с окружающей средой. На каждом шаге агент получает состояние среды, выбирает действие, получает вознаграждение и переходит в новое состояние. Цель агента состоит в формировании такой стратегии поведения, которая позволяет максимизировать суммарное ожидаемое вознаграждение.

В данной лабораторной работе рассматриваются алгоритмы семейства Actor-Critic. Эти алгоритмы объединяют два подхода к обучению с подкреплением: методы на основе стратегии и методы на основе функции ценности. Компонент Actor отвечает за выбор действий и формирование стратегии поведения агента, а компонент Critic оценивает качество выбранных действий с помощью функции ценности состояния.

Для проведения экспериментов была выбрана среда CartPole-v1 из библиотеки Gymnasium. В данной среде агент управляет тележкой и должен удерживать шест в вертикальном положении как можно дольше. Для решения задачи была реализована нейронная сеть Actor-Critic, которая одновременно формирует вероятности действий и оценивает ценность текущего состояния.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо реализовать любой алгоритм семейства Actor-Critic для произвольной среды обучения с подкреплением. Алгоритмы данного семейства используют одновременно две модели: Actor и Critic. Actor отвечает за выбор действия, а Critic оценивает текущее состояние и помогает корректировать стратегию агента.

В данной работе был реализован базовый алгоритм Actor-Critic для среды CartPole-v1. Эта среда имеет непрерывное пространство наблюдений, состоящее из 4 признаков, и дискретное пространство действий, содержащее 2 возможных действия. Такая структура

хорошо подходит для реализации нейронной сети, которая обрабатывает состояние среды и формирует стратегию поведения агента.

В ходе выполнения лабораторной работы необходимо:

- импортировать необходимые библиотеки;
- создать среду CartPole-v1 и вывести ее основные характеристики;
- реализовать нейронную сеть Actor-Critic;
- создать агента Actor-Critic;
- выполнить обучение агента;
- проверить качество обученного агента на тестовых эпизодах;
- построить график наград при обучении;
- сформировать итоговую таблицу результатов.

3) Текст программы и экранные формы с примерами выполнения

В данном разделе приведены основные этапы программной реализации алгоритма Actor-Critic для среды CartPole-v1. Все вычисления были выполнены в среде Google Colab с использованием языка программирования Python и графического ускорителя CUDA.

Для работы со средой обучения с подкреплением использовалась библиотека Gymnasium. Для численных вычислений применялась библиотека numpy, для формирования итоговой таблицы — pandas, а для построения графиков — matplotlib. Для отображения процесса обучения использовалась библиотека tqdm.

Для построения и обучения нейронной сети использовалась библиотека PyTorch. В реализации применялась общая нейронная сеть Actor-Critic: общие скрытые слои выделяют признаки из состояния среды, Actor формирует вероятности действий, а Critic оценивает ценность текущего состояния.

```
▶ # Импорт необходимых библиотек для выполнения лабораторной работы

try:
    import gymnasium as gym
except ImportError:
    !pip install gymnasium -q
    import gymnasium as gym

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import time
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions import Categorical

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Библиотеки успешно импортированы")
print("Версия Gymnasium:", gym.__version__)
print("Версия NumPy:", np.__version__)
print("Версия PyTorch:", torch.__version__)
print("Используемое устройство:", device)
```

```
... Библиотеки успешно импортированы
Версия Gymnasium: 1.3.0
Версия NumPy: 2.0.2
Версия PyTorch: 2.11.0+cu128
Используемое устройство: cuda
```

Рисунок 1 — Импорт необходимых библиотек

3.1. Создание среды CartPole-v1 и вывод ее характеристик

На первом этапе была создана среда CartPole-v1 из библиотеки Gymnasium. Данная среда моделирует задачу управления тележкой с закрепленным на ней шестом. Цель агента состоит в том, чтобы перемещать тележку влево или вправо и удерживать шест в вертикальном положении как можно дольше.

После создания среды были выведены ее основные характеристики. Пространство наблюдений является непрерывным и содержит 4 признака состояния: положение тележки, скорость тележки, угол наклона шеста и угловую скорость шеста. Пространство действий является дискретным и содержит 2 возможных действия.

Полученная информация показывает, что среда CartPole-v1 хорошо подходит для реализации алгоритма Actor-Critic. Нейронная сеть может использовать 4 признака состояния в качестве входных данных и формировать вероятности выбора двух возможных действий.



Создание среды CartPole-v1 и вывод основных характеристик

```
env = gym.make("CartPole-v1")

print("Среда:", "CartPole-v1")

print("\nПространство наблюдений:")
print(env.observation_space)

print("\nПространство действий:")
print(env.action_space)

print("\nКоличество признаков состояния:", env.observation_space.shape[0])
print("Количество действий:", env.action_space.n)

state, info = env.reset()

print("\nНачальное состояние:")
print(state)

print("\nОписание признаков состояния:")
print("1. Положение тележки")
print("2. Скорость тележки")
print("3. Угол наклона шеста")
print("4. Угловая скорость шеста")
```

Рисунок 2 — Код создания среды CartPole-v1

```

... Среда: CartPole-v1

Пространство наблюдений:
Box([-4.8          -inf -0.41887903      -inf], [4.8          inf 0.41887903      inf], (4,), float32)

Пространство действий:
Discrete(2)

Количество признаков состояния: 4
Количество действий: 2

Начальное состояние:
[ 0.03655644 -0.01352966 -0.02239741  0.02603528]

Описание признаков состояния:
1. Положение тележки
2. Скорость тележки
3. Угол наклона шеста
4. Угловая скорость шеста

```

Рисунок 3 — Результат создания среды CartPole-v1 и вывод характеристик

3.2. Создание нейронной сети Actor-Critic

После создания среды была реализована нейронная сеть Actor-Critic. Данная сеть объединяет два компонента: Actor и Critic. Actor отвечает за формирование стратегии поведения агента, то есть вычисляет вероятности выбора возможных действий. Critic оценивает ценность текущего состояния и используется для корректировки обучения Actor.

Входом нейронной сети является состояние среды CartPole-v1, состоящее из 4 числовых признаков. Сначала состояние обрабатывается общими полносвязными слоями, которые выделяют признаки. Затем результат передается в две отдельные части сети. Первая часть, Actor, формирует вероятности двух действий с помощью функции Softmax. Вторая часть, Critic, возвращает одно число — оценку ценности состояния.

После создания сети был выполнен тестовый прямой проход для начального состояния среды. В результате были получены вероятности выбора действий и оценка ценности состояния. Это подтверждает, что сеть корректно принимает состояние среды и формирует оба необходимых выхода для алгоритма Actor-Critic.



Создание нейронной сети Actor-Critic

```
class ActorCriticNetwork(nn.Module):
    """
    Нейронная сеть Actor-Critic.
    Actor формирует вероятности действий.
    Critic оценивает ценность состояния.
    """

    def __init__(self, state_dim, action_dim):
        super(ActorCriticNetwork, self).__init__()

        self.shared_layers = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU()
        )

        self.actor = nn.Sequential(
            nn.Linear(128, action_dim),
            nn.Softmax(dim=-1)
        )

        self.critic = nn.Linear(128, 1)

    def forward(self, state):
        """
        Прямой проход сети.
        """
        features = self.shared_layers(state)
        action_probs = self.actor(features)
        state_value = self.critic(features)
        return action_probs, state_value

state_dim = env.observation_space.shape[0]
action_dim = env.action_space.n

actor_critic_net = ActorCriticNetwork(state_dim, action_dim).to(device)

sample_state = torch.tensor(state, dtype=torch.float32).unsqueeze(0).to(device)
sample_action_probs, sample_state_value = actor_critic_net(sample_state)

print("Нейронная сеть Actor-Critic успешно создана")
print("Количество признаков состояния:", state_dim)
print("Количество действий:", action_dim)
print("Размер входного состояния:", sample_state.shape)
print("Вероятности действий:", sample_action_probs.detach().cpu().numpy())
print("Оценка ценности состояния:", sample_state_value.item())

print("\nАрхитектура сети:")
print(actor_critic_net)
```

Рисунок 4 — Код создания нейронной сети Actor-Critic

```

... Нейронная сеть Actor-Critic успешно создана
Количество признаков состояния: 4
Количество действий: 2
Размер входного состояния: torch.Size([1, 4])
Вероятности действий: [[0.5715286 0.4284714]]
Оценка ценности состояния: -0.12876655161380768

Архитектура сети:
ActorCriticNetwork(
  (shared_layers): Sequential(
    (0): Linear(in_features=4, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=128, bias=True)
    (3): ReLU()
  )
  (actor): Sequential(
    (0): Linear(in_features=128, out_features=2, bias=True)
    (1): Softmax(dim=-1)
  )
  (critic): Linear(in_features=128, out_features=1, bias=True)
)

```

Рисунок 5 — Результат создания нейронной сети Actor-Critic

3.3. Создание агента Actor-Critic

На следующем этапе был создан агент Actor-Critic, который использует построенную нейронную сеть для взаимодействия со средой CartPole-v1. Агент содержит модель, оптимизатор и параметры обучения, необходимые для обновления весов нейронной сети.

Для выбора действия агент использует вероятности, сформированные компонентом Actor. На основе этих вероятностей создается категориальное распределение, из которого выбирается действие. Такой подход позволяет агенту исследовать разные варианты поведения, а не всегда выбирать только действие с максимальной вероятностью.

Компонент Critic используется для оценки ценности текущего состояния. Эта оценка необходима для вычисления преимущества *advantage*, которое показывает, насколько выбранное действие оказалось лучше или хуже ожидаемого результата. Полученное значение используется при обновлении Actor и Critic в процессе обучения.



Создание агента Actor-Critic

```
class ActorCriticAgent:
    """
    Агент Actor-Critic для обучения в среде CartPole-v1.
    """

    def __init__(self, env, model, gamma=0.99, lr=0.001):
        self.env = env
        self.model = model
        self.gamma = gamma

        self.optimizer = optim.Adam(self.model.parameters(), lr=lr)

        self.episodes_reward = []
        self.episodes_steps = []
        self.actor_losses = []
        self.critic_losses = []

    def select_action(self, state):
        """
        Выбор действия на основе вероятностей, сформированных Actor.
        """
        state_tensor = torch.tensor(state, dtype=torch.float32).unsqueeze(0).to(device)

        action_probs, state_value = self.model(state_tensor)
        distribution = Categorical(action_probs)
        action = distribution.sample()

        log_prob = distribution.log_prob(action)

        return action.item(), log_prob, state_value

    def print_info(self):
        """
        Вывод основной информации об агенте.
        """
        print("Агент Actor-Critic успешно создан")
        print("Среда:", "CartPole-v1")
        print("Количество признаков состояния:", self.env.observation_space.shape[0])
        print("Количество действий:", self.env.action_space.n)
        print("Gamma:", self.gamma)
        print("Оптимизатор:", self.optimizer)

ac_agent = ActorCriticAgent(env, actor_critic_net)

ac_agent.print_info()
```

Рисунок 6 — Код создания агента Actor-Critic

```
... Агент Actor-Critic успешно создан
Среда: CartPole-v1
Количество признаков состояния: 4
Количество действий: 2
Gamma: 0.99
Оптимизатор: Adam (
Parameter Group 0
  amsgrad: False
  betas: (0.9, 0.999)
  capturable: False
  decoupled_weight_decay: False
  differentiable: False
  eps: 1e-08
  foreach: None
  fused: None
  lr: 0.001
  maximize: False
  weight_decay: 0
)
```

Рисунок 7 — Результат инициализации агента Actor-Critic

3.4. Обучение агента Actor-Critic

После создания агента было выполнено обучение алгоритма Actor-Critic в среде CartPole-v1. В процессе обучения агент взаимодействовал со средой, выбирал действия на основе вероятностей, сформированных Actor, получал вознаграждения и обновлял параметры нейронной сети.

На каждом шаге вычислялось целевое значение TD-target, основанное на текущей награде и оценке ценности следующего состояния. Затем вычислялось преимущество advantage как разность между целевым значением и текущей оценкой Critic. Это значение использовалось для обновления двух частей сети: Actor обучался выбирать более выгодные действия, а Critic обучался точнее оценивать ценность состояний.

Обучение выполнялось в течение 500 эпизодов. В результате обучение было завершено за 245.323019 секунды. Средняя награда за последние 50 эпизодов составила 250.1, среднее количество шагов за последние 50 эпизодов также составило 250.1. Максимальная награда при обучении достигла 500.0, что соответствует максимальному значению для среды CartPole-v1.

```

# Обучение агента Actor-Critic

def train_actor_critic(agent, num_episodes=500, max_steps=500):
    """
    Обучение агента Actor-Critic в среде CartPole-v1.
    """
    start_time = time.time()

    for episode in tqdm(range(num_episodes)):
        state, info = agent.env.reset()

        total_reward = 0
        steps = 0
        done = False

        while not done and steps < max_steps:
            action, log_prob, state_value = agent.select_action(state)

            next_state, reward, terminated, truncated, info = agent.env.step(action)
            done = terminated or truncated

            next_state_tensor = torch.tensor(next_state, dtype=torch.float32).unsqueeze(0).to(device)

            with torch.no_grad():
                _, next_state_value = agent.model(next_state_tensor)

            reward_tensor = torch.tensor([[reward]], dtype=torch.float32).to(device)
            done_tensor = torch.tensor([[done]], dtype=torch.float32).to(device)

            td_target = reward_tensor + agent.gamma * next_state_value * (1 - done_tensor)
            advantage = td_target - state_value

            actor_loss = -log_prob * advantage.detach()
            critic_loss = advantage.pow(2)
            total_loss = actor_loss + critic_loss

            agent.optimizer.zero_grad()
            total_loss.backward()
            agent.optimizer.step()

            agent.actor_losses.append(actor_loss.item())
            agent.critic_losses.append(critic_loss.item())

            state = next_state
            total_reward += reward
            steps += 1

        agent.episodes_reward.append(total_reward)
        agent.episodes_steps.append(steps)

        if (episode + 1) % 50 == 0:
            print(
                "Эпизод:", episode + 1,
                "| Награда:", total_reward,
                "| Шаги:", steps,
                "| Средняя награда за последние 50 эпизодов:",
                np.mean(agent.episodes_reward[-50:])
            )

    training_time = time.time() - start_time

    print("\nОбучение Actor-Critic завершено")
    print("Количество эпизодов:", num_episodes)
    print("Время обучения:", training_time)
    print("Средняя награда за последние 50 эпизодов:", np.mean(agent.episodes_reward[-50:]))
    print("Среднее количество шагов за последние 50 эпизодов:", np.mean(agent.episodes_steps[-50:]))
    print("Максимальная награда:", np.max(agent.episodes_reward))

    return training_time

training_time = train_actor_critic(ac_agent, num_episodes=500, max_steps=500)

```

Рисунок 8 — Код обучения агента Actor-Critic

Обучение Actor-Critic завершено
Количество эпизодов: 500
Время обучения: 245.32301926612854
Средняя награда за последние 50 эпизодов: 250.1
Среднее количество шагов за последние 50 эпизодов: 250.1
Максимальная награда: 500.0

Рисунок 9 — Результат обучения агента Actor-Critic

3.5. Проверка обученного агента Actor-Critic

После завершения обучения была выполнена проверка обученного агента Actor-Critic на тестовых эпизодах. На данном этапе агент больше не выбирал действия случайным образом из распределения, а использовал жадный выбор действия с максимальной вероятностью, сформированной компонентом Actor.

Проверка выполнялась на 20 тестовых эпизодах. Для каждого эпизода фиксировались суммарная награда и количество шагов до завершения. Такой подход позволяет оценить качество полученной стратегии отдельно от процесса обучения.

По результатам проверки средняя тестовая награда составила 211.3, а среднее количество шагов — 211.3. Минимальная награда составила 198.0, максимальная награда — 220.0. Полученные результаты показывают, что обученный агент способен достаточно устойчиво удерживать шест в вертикальном положении в среде CartPole-v1.



Проверка обученного агента Actor-Critic

```
def evaluate_actor_critic(agent, episodes_count=20, max_steps=500):
    """
    Проверка качества обученного агента Actor-Critic.
    """
    test_rewards = []
    test_steps = []

    for episode in range(episodes_count):
        state, info = agent.env.reset()

        total_reward = 0
        steps = 0
        done = False

        while not done and steps < max_steps:
            state_tensor = torch.tensor(state, dtype=torch.float32).unsqueeze(0).to(device)

            with torch.no_grad():
                action_probs, state_value = agent.model(state_tensor)
                action = torch.argmax(action_probs).item()

            next_state, reward, terminated, truncated, info = agent.env.step(action)
            done = terminated or truncated

            state = next_state
            total_reward += reward
            steps += 1

        test_rewards.append(total_reward)
        test_steps.append(steps)

    print(
        "Тестовый эпизод:", episode + 1,
        "| Награда:", total_reward,
        "| Шаги:", steps
    )

    print("\nПроверка обученного агента Actor-Critic завершена")
    print("Количество тестовых эпизодов:", episodes_count)
    print("Средняя тестовая награда:", np.mean(test_rewards))
    print("Среднее количество шагов:", np.mean(test_steps))
    print("Минимальная награда:", np.min(test_rewards))
    print("Максимальная награда:", np.max(test_rewards))

    return test_rewards, test_steps

test_rewards, test_steps = evaluate_actor_critic(ac_agent, episodes_count=20, max_steps=500)
```

Рисунок 10 — Код проверки обученного агента Actor-Critic

...	Тестовый эпизод: 1		Награда: 218.0		Шаги: 218
	Тестовый эпизод: 2		Награда: 218.0		Шаги: 218
	Тестовый эпизод: 3		Награда: 218.0		Шаги: 218
	Тестовый эпизод: 4		Награда: 220.0		Шаги: 220
	Тестовый эпизод: 5		Награда: 215.0		Шаги: 215
	Тестовый эпизод: 6		Награда: 207.0		Шаги: 207
	Тестовый эпизод: 7		Награда: 206.0		Шаги: 206
	Тестовый эпизод: 8		Награда: 210.0		Шаги: 210
	Тестовый эпизод: 9		Награда: 205.0		Шаги: 205
	Тестовый эпизод: 10		Награда: 198.0		Шаги: 198
	Тестовый эпизод: 11		Награда: 206.0		Шаги: 206
	Тестовый эпизод: 12		Награда: 213.0		Шаги: 213
	Тестовый эпизод: 13		Награда: 217.0		Шаги: 217
	Тестовый эпизод: 14		Награда: 212.0		Шаги: 212
	Тестовый эпизод: 15		Награда: 210.0		Шаги: 210
	Тестовый эпизод: 16		Награда: 208.0		Шаги: 208
	Тестовый эпизод: 17		Награда: 210.0		Шаги: 210
	Тестовый эпизод: 18		Награда: 219.0		Шаги: 219
	Тестовый эпизод: 19		Награда: 200.0		Шаги: 200
	Тестовый эпизод: 20		Награда: 216.0		Шаги: 216

Проверка обученного агента Actor-Critic завершена

Количество тестовых эпизодов: 20

Средняя тестовая награда: 211.3

Среднее количество шагов: 211.3

Минимальная награда: 198.0

Максимальная награда: 220.0

Рисунок 11 — Результат проверки обученного агента Actor-Critic

3.6. Построение графика наград при обучении Actor-Critic

Для наглядного анализа процесса обучения был построен график суммарных наград, полученных агентом Actor-Critic в каждом эпизоде. По оси абсцисс отложен номер эпизода, а по оси ординат — суммарная награда за эпизод.

Дополнительно на графике было построено скользящее среднее за 20 эпизодов. Этот показатель позволяет сгладить случайные колебания наград и более наглядно оценить общую динамику обучения агента.

Из графика видно, что в начале обучения награды имеют невысокие значения, так как агент еще не сформировал эффективную стратегию поведения. По мере обучения награды постепенно увеличиваются, а в отдельных эпизодах достигают значения 500. Это показывает, что реализованный алгоритм Actor-Critic позволяет агенту улучшать стратегию управления тележкой в среде CartPole-v1.



Рисунок 12 — График наград при обучении Actor-Critic

3.7. Формирование итоговой таблицы результатов

На заключительном этапе выполнения программы была сформирована итоговая таблица результатов лабораторной работы. В таблицу были включены основные характеристики проведенного эксперимента: выбранная среда, реализованный алгоритм, количество признаков состояния, количество действий, количество эпизодов обучения, время обучения, средняя награда за последние 50 эпизодов, максимальная награда при обучении, а также результаты тестирования обученного агента.

Итоговая таблица показывает, что в работе использовалась среда CartPole-v1 и алгоритм Actor-Critic. Пространство состояния содержит 4 признака, а пространство действий содержит 2 возможных действия. Обучение агента выполнялось в течение 500 эпизодов и заняло 245.323019 секунды.

По результатам обучения средняя награда за последние 50 эпизодов составила 250.1, а максимальная награда при обучении достигла 500.0. На этапе тестирования агент был проверен на 20 эпизодах. Средняя тестовая награда составила 211.3, минимальная тестовая награда — 198.0, а максимальная тестовая награда — 220.0.

	Показатель	Значение
0	Среда	CartPole-v1
1	Алгоритм	Actor-Critic
2	Количество признаков состояния	4
3	Количество действий	2
4	Количество эпизодов обучения	500
5	Время обучения	245.323019
6	Средняя награда за последние 50 эпизодов	250.1
7	Среднее количество шагов за последние 50 эпизодов	250.1
8	Максимальная награда при обучении	500.0
9	Количество тестовых эпизодов	20
10	Средняя тестовая награда	211.3
11	Среднее количество шагов при тестировании	211.3
12	Минимальная тестовая награда	198.0
13	Максимальная тестовая награда	220.0

Рисунок 13 — Итоговая таблица результатов лабораторной работы

4) Заключение

В ходе лабораторной работы был изучен и реализован алгоритм семейства Actor-Critic для задачи обучения с подкреплением. В качестве среды была выбрана CartPole-v1 из библиотеки Gymnasium. Данная среда имеет непрерывное пространство наблюдений из 4 признаков и дискретное пространство действий из 2 возможных действий, что делает ее удобной для реализации нейросетевого алгоритма Actor-Critic.

В процессе выполнения работы была создана нейронная сеть Actor-Critic, состоящая из общих скрытых слоев и двух выходных частей. Компонент Actor формировал вероятности выбора действий, а компонент Critic оценивал ценность текущего состояния. Для обучения использовалось преимущество advantage, вычисляемое на основе TD-target и оценки Critic.

Обучение агента выполнялось в течение 500 эпизодов. Средняя награда за последние 50 эпизодов составила 250.1, а максимальная награда при обучении достигла 500.0. После обучения агент был протестирован на 20 эпизодах. Средняя тестовая награда составила 211.3, минимальная награда — 198.0, максимальная награда — 220.0.

Полученные результаты показывают, что реализованный алгоритм Actor-Critic позволил агенту сформировать эффективную стратегию управления тележкой в среде CartPole-v1. Таким образом, все поставленные задачи лабораторной работы были выполнены: была создана среда, реализована нейронная сеть Actor-Critic, выполнено обучение агента, проведена проверка качества полученной стратегии и сформирована итоговая таблица результатов.